

Large Language Model guided State Selection Approach for Fuzzing Network Protocol

1st Bo Yu^a, 1st Qihong Song^{a*}, 2nd Chengnuo Cai^{a*}

^a School of Computer, National University of Defense Technology, Changsha, China

yubo0615@nudt.edu.cn, hawkeysong@163.com, caichengnuo277@nudt.edu.cn

Abstract—Fuzzing network protocols is challenging due to their various factors including protocol state and state transitions. To achieve better state coverage when fuzzing network protocol with grey-box fuzzing, several approaches are proposed to select valuable states and optimize the fuzzing process. Based on the ability of extensive knowledge integration and reasoning, large language models (LLMs) are also imported to generate more effective test cases for protocol fuzzing. However, these approaches leave poor state and code coverage on real-world services protocol evaluation since they use either random selection or heuristics.

To address this issue, we present LLMgSSA, a Large Language Model guided State Selection Approach, which navigates the LLM to reason about protocol state selection. In the approach, LLMgSSA first extracts the features of the current protocol and state space and determines valuable states by interacting with the LLM. It then collects and analyzes the current status of each covered state and combines the inference results of the LLM for the final state selection. To evaluate the effectiveness of LLMgSSA, we have conducted extensive experiments with five real-world protocols from ProFuzzBench. Experimental results show that, compared to three state-of-the-art fuzzers, CHATAFL, AFLNET, and NSFuzz, LLMgSSA can increase state transitions, covered states, branch coverage, and line coverage by up to 70.6%, 35.3%, 13.1%, and 13.1% respectively.

Index Terms—network protocol software, fuzz testing, large language model, prompt engineering

I. INTRODUCTION

In real-world network communications, network protocol software implements the rules for information exchange between entities as defined by the protocol. Due to characteristics such as exposed interfaces during operation, complex implementations, large-scale deployment, and concealed causes of vulnerabilities, network protocol software is more vulnerable to widespread attacks. Fuzzing is an effective automated technique for testing network protocol software. It automatically generates numerous test cases, systematically exploring protocol boundary conditions and error-handling capabilities, making it especially effective at detecting network protocol vulnerabilities.

Most protocols maintain a state of connection between the client and the server during communication, known as a stateful protocol. These protocols possess a broad state space accompanied by complex state transitions. State selection is one of the core strategies to ensure effective fuzzing, requiring the selection based on complex state transitions and conditional dependencies to guide protocol fuzzing [1], [2]. Rational

state selection can make fuzzing more goal-oriented, covering more states and code paths, avoiding excessive testing of stable states, and focusing more on potential weak points.

Current protocol fuzzers mainly utilize random selection approaches or heuristic algorithms based on limited state feedback [2]–[5], which fail to select valuable states to guide the fuzzing process effectively. There are several challenges in state selection: **(C1)** The construction of state spaces and state transitions are based on protocol rules, and specific protocol features need to be considered during state selection. **(C2)** The covered states during the fuzzing provide rich feedback information, which can assist in identifying potentially valuable states. **(C3)** In complex scenarios of protocol fuzzing, state selection requires enhanced analytical and reasoning capabilities.

CHATAFL [4] is a protocol-fuzzing framework that intelligently generates effective test cases through interaction with LLMs. Leveraging LLMs' knowledge and reasoning capabilities about protocol, CHATAFL expands the initial seed corpus, conducts structure-aware mutation based on the extracted protocol grammar, and generates messages to break out of the coverage plateau. However, CHATAFL still uses the basic heuristic algorithm from AFLNET and has not explored the potential benefits of LLMs in state selection.

To alleviate these challenges, we explore the potential of LLMs in state selection for fuzzing, specifically verifying LLM's mastery of protocol state spaces and their reasoning and construction capabilities with specific protocol state machines. Subsequently, we propose an LLM-guided state selection approach for protocol fuzzing, named LLMgSSA. The approach first collects the protocol and state information of the current fuzzing. Secondly, based on the features of the current protocol and state space, valuable states are determined by interacting with LLM by prompt engineering (for **C1**). It also evaluates the value of each state by analyzing the multidimensional fuzzing feedback on covered states (for **C2**). Finally, it identifies the optimal state by combining the reasoning results of LLM with the performance feedback of fuzzing (for **C3**).

To evaluate the effectiveness of LLMgSSA in enhancing the performance of protocol fuzzing, we selected five protocol implementation software from the stateful protocol fuzzing benchmark ProFuzzBench [6] for our experiments. Compared to the CHATAFL, LLMgSSA increased the covered states, state transitions, branch coverage, and line coverage by 2.7%,

Bo Yu and Qihong Song contributed equally to this work.

* is the corresponding author.

12.1%, 6.5%, and 4.9%, respectively. Compared to two other advanced baseline fuzzers, AFLNET and NSFuzz, LLMgSSA achieved higher improvements on these metrics.

The contributions of this work can be summarized as follows

- We propose LLMgSSA for LLM-based state selection in stateful protocol fuzzing. This approach intelligently selects valuable states based on the features of current protocols and the feedback information of each state in the current state space, covering more states and code, which helps discover more potential vulnerabilities.
- We implemented LLMgSSA based on ChatAFL, where the state selection component comprises a perception module, an intelligent reasoning module, and a comprehensive decision-making module. We will release the source code of LLMgSSA at <https://github.com/ipasslab/LLMgSSA>.
- We conducted extensive experiments to validate the superiority of LLMgSSA in protocol fuzzing. The experimental results demonstrate that, compared to three state-of-the-art baseline fuzzers—CHATAFL, AFLNET, and NSFuzz—LLMgSSA significantly improves state transitions, state coverage, branch coverage, and line coverage.

II. CASE STUDY: TEST THE ABILITY OF LLMs TO SELECT VALUABLE STATES.

This section tests the implementation of the Real Time Streaming Protocol (RTSP), Live555. We employ the ChatGPT-3.5 model to assess LLM’s mastery of the specific protocol state space and its capability to construct a state machine. These capabilities significantly indicate the LLMs’ ability to perform effective state selection.

A. Mastery of Protocol State Space by LLMs.

In protocol fuzzing, the fuzzing tool acts as the client, and the target protocol software serves as the server. Messages from the client are referred to as requests, while messages from the server are responses. For each request message from the client, the server replies with a response message that includes a status code. All status codes form the state space of the protocol.

We asked the LLM to provide all possible status codes returned by Live555, verifying its mastery of the RTSP status space using the authoritative document Request for Comments (RFC) 2326. According to RFC 2326, there are 45 different status codes, including one informational status code (100), three successful status codes (2XX), six redirection status codes (3XX), 28 client error status codes (4XX), and seven server error status codes (5XX).

We queried the LLM 50 times about all possible Live555 status codes and analyzed the responses. The results showed that out of 50 responses, the LLM missed status code 304 twice, status code 501 once, and incorrectly provided status code 101 once. Therefore, there was an average of only about 0.08 errors per interaction for all results of 45 status codes,

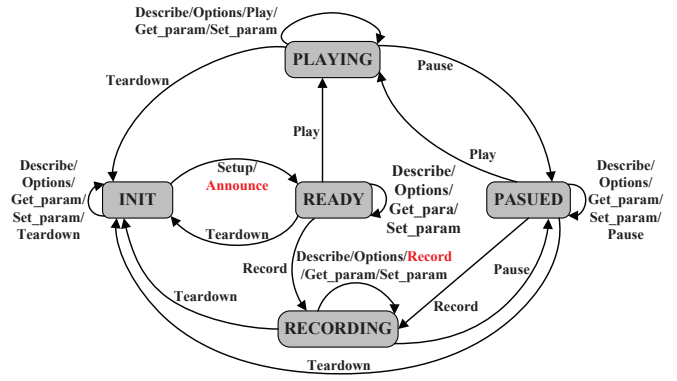


Fig. 1. The state machine for the RTSP protocol from LLM and RFC 2326.

indicating that the LLM effectively comprehends the complex status space of specific protocols.

B. The Capability of LLMs to Construct State Machine.

A state machine is a model used to describe network protocols’ operational logic and behaviour. It represents the states a protocol might reach during its lifecycle and the state transitions that occur upon receiving specific requests. It should be noted that the states described here refer to different phases in the lifecycle of a protocol session. The status codes mentioned above are part of the response message and are used to indicate the outcome of a specific request. If an LLM can successfully construct an accurate state machine model, it not only demonstrates its deep understanding of the specific protocol’s workings, operational logic, and behavioural details but also reflects the LLM’s ability to integrate protocol knowledge and perform logical reasoning.

We queried the LLM 10 times about possible state transitions within the RTSP state machine and compiled the results into the state machine shown in Fig. 1 with black lines and text. The figure displays the state transitions of RTSP under different states in response to various requests. Based on the document RFC 2326, we also created a complete state machine for RTSP, as shown by all the lines and text in Fig. 1. From the Fig. 1, it is evident that, except for two transitions indicated by red text that the LLM failed to describe, all other state transitions were correctly depicted. These results indicate that the LLM has a relatively deep understanding of the protocol and can basically accurately construct specific state machines.

III. LLM GUIDED STATE SELECTION APPROACH

The case study has demonstrated that LLMs have a deep understanding of the protocol’s state space and the ability to reason about protocol state transitions. The results indicate that LLMs are capable of guiding fuzzers in selecting valuable states. Therefore, we propose the LLM-guided state selection approach, LLMgSSA, which can intelligently select valuable states based on specific protocol expertise and current test feedback.

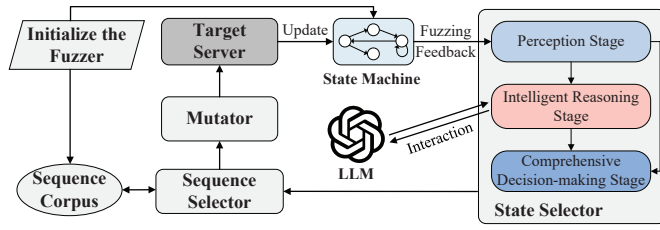


Fig. 2. The framework of LLMgSSA.

The framework of LLMgSSA for fuzzing is shown in Fig. 2. After initializing the fuzzer, including the state machine and sequence (seed) corpus, the fuzzing process begins its iterations. Firstly, the fuzzer undergoes three stages to choose a valuable state. In the perception stage, valuable states are determined based on multidimensional state feedback. In the intelligent reasoning stage, valuable states are inferred through interaction with LLM using current protocol and state information via prompt engineering. The comprehensive decision-making stage selects the final state based on the analysis results from the first two stages. Next, the sequence selector selects sequences from the sequence corpus based on the selected states. The chosen sequences undergo mutation to generate test cases, which are then sent to the target server. Finally, the state machine and feedback information are updated based on the server's response, preparing for the next round of fuzzing.

The three-stage state selection process of LLMgSSA is detailed in Algorithm 1.

A. Perception Stage. (Lines 1-5 of Alg.1)

During the fuzzing, feedback information about the state plays a crucial role in guiding the fuzzing. This information provides feedback on fuzzing performance. The richer the state feedback, the more comprehensive the description of the current fuzzing performance, and the more accurate the judgment of valuable states. Current heuristic algorithms only use three types of state information: the number of times fuzzed, the number of times selected, and the number of new paths discovered when the state is selected. In fact, the total number of paths that exist in that state and the total number of seeds related to that state also play an important role in evaluating the value of states in guiding fuzzing. LLMgSSA's perception stage evaluates each state's value by collecting and analyzing five types of feedback information about the states. Here are the five types of feedback information and their significance for value assessment.

a) *The total number of times the state has been fuzzed (fuzz_num):* Fuzz_num records the actual number of fuzzing performed based on this state, reflecting the state's activity level in actual fuzzing. A high fuzz_num value indicates that the state has been frequently selected and used to generate new test cases.

b) *The total number of times the state has been selected (selected_num):* Selected_num records the number of times

Algorithm 1 State Selection Algorithm Based on LLM

Input: P : protocol implementation; F_s : current status of fuzzing; M : state machine.

Output: the final selected state S

```

1: State set  $X < s_1, s_2, \dots, s_N > \leftarrow \text{ListAllStates}(M)$ 
2: for  $s_i$  in  $X$  do
3:    $path\_cover\_num, selected\_num, path\_disc\_num,$ 
    $seed\_num, fuzz\_num \leftarrow \text{CollectFeedback}(F_s, s_i)$ 
4:    $Sensing\_Score_i = \text{Score}(fuzz\_num, selected\_num,$ 
    $path\_disc\_num, seed\_num, path\_cover\_num)$ 
5: end for
6: Prompt  $Pr \leftarrow \text{ConstructPrompt}(P, X)$ 
7: Answer  $A \leftarrow \text{ChatwithLLM}(Pr)$ 
8: Valuable state set  $V \leftarrow \text{Extractor}(A)$ 
9: for  $i$  from 1 to  $N$  do
10:  if  $s_i$  in  $V$  then
11:     $exit = 1$ 
12:  else
13:     $exit = 0$ 
14:  end if
15: end for
16:  $State\_Score_i = Sensing\_Score_i(1 + 0.2 \cdot exit)$ 
17: # Select a state with a higher score with a higher probability randomly.
18:  $S \leftarrow \text{RandomSelect}(State\_Score_1, \dots, State\_Score_N)$ 

```

a state has been chosen as the starting point for testing during the fuzzing process. A high *selected_num* value indicates that the code paths associated with this state have likely been thoroughly tested.

c) *The number of new paths discovered when the state was selected (path_disc_num):* Path_disc_num provides a direct indicator of whether fuzzing starting from this state effectively increased code coverage or discovered new program behaviours. A high path_disc_num indicates that this state is an effective starting point for exploring unknown program behaviours.

d) *The total number of paths that reach the state (path_cover_num):* Path_cover_num records the number of all possible paths that lead to the current state. A high path_cover_num indicates the centrality of this state in the program, suggesting that new or insufficiently tested areas of code might be discovered through this state.

e) *The number of seeds related to the state (seed_num):* Seed_num refers to the number of test cases that can trigger this state. A high seed_num indicates that more test cases can be generated from this state, suggesting that this state may have higher complexity and centrality in the program.

In the perception stage of Algorithm 1, we collect the above five-dimensional feature information of covered states and integrates all the information to score the value of all states. Specifically, fuzz_num and selected_num are inversely proportional to the state value, while the other three types of information are directly proportional to the state value.

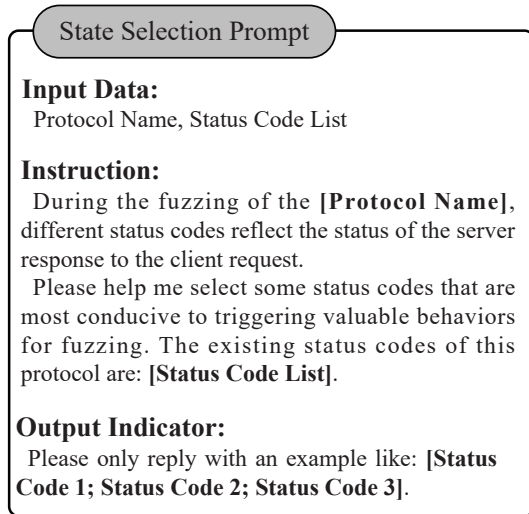


Fig. 3. Prompt for intelligent reasoning stage.

B. Intelligent Reasoning Stage. (Lines 6-8 of Alg.1)

We would like to utilize LLM to guide the state selection in fuzzing. Interacting directly with LLMs often results in inaccurate or low-quality responses, and LLMs may even generate redundant or irrelevant data. Clear, specific questions and sufficient background information can help LLMs understand users' needs more accurately. Prompt engineering is the process of carefully designing and optimizing inputs during interactions with LLMs. This engineering aims to maximize the quality and relevance of the model's output, ensuring that users receive the most accurate and useful responses.

For our intelligent reasoning stage, we designed prompts that build detailed and specific contextual information by inputting current fuzzing-related information, generating concise and clear instructions, and specifying output prompts to clarify the output format, as shown in Fig. 3. Specifically, the specific protocol implementation type and the currently covered states in fuzzing affect the state selection results, so we use these two pieces of information as variable input information for the prompts, collecting corresponding information to construct instructions before each state selection. When constructing instructions, we also supplemented the concept of state, the purpose of state selection, and other relevant contextual information, aiming to enable the LLM to reason based on context knowledge. Finally, we specified the format of LLM's responses, which helps ensure the accuracy of the answers, reduce redundancy, and facilitate the extraction of key information.

In the intelligent reasoning stage of Algorithm 1, we first constructs the prompts and interacts with the LLM to obtain a list of valuable states. We then traverse all covered states and determines whether the current state is valuable, as identified by LLM reasoning, assigning a higher score to the valuable states.

C. Comprehensive Decision-making Stage. (Lines 9-18 of Alg.1)

The perception stage focuses on real-time fuzzing results and statistics, providing direct, performance-based feedback. The intelligent reasoning stage utilizes the intelligence of LLM, focusing on prediction and theoretical analysis to assess the value of states in complex scenarios. The combination of these two approaches enhances the state selection strategy from different perspectives, increasing the comprehensiveness and depth of testing.

In the comprehensive decision-making stage of Algorithm 1, we combine the scores of each state from the perception stage and the intelligent reasoning stage to assign a final value score to each covered state. We select states with higher value scores with a higher probability. At the same time, there is also a small probability of selecting states with lower value scores. This is designed to maintain the randomness of state selection, avoid limitations, and increase exploratory potential.

D. Implementation.

We have implemented LLMgSSA based on the advanced protocol fuzzer CHATAFL. CHATAFL is a fuzzing tool based on AFL (American Fuzzy Lop) [7] that obtains protocol grammar through interaction with LLM, uses the grammar to enhance seeds and guide mutations, and can also use interactive information to generate messages to break through coverage bottlenecks directly, achieving the best fuzzing effects. Based on CHATAFL's foundational framework and optimization methods, we expanded the state selection component. LLMgSSA specifically integrates three modules: a perception module, an intelligent reasoning module, and a comprehensive decision-making module, achieving intelligent state selection and optimizing the fuzzing process.

IV. EXPERIMENTS AND EVALUATION

To evaluate the fuzzing effectiveness of LLMgSSA, we conducted five 24-hour fuzzing sessions using LLMgSSA along with three advanced baseline fuzzers—CHATAFL, NSFuzz, and AFLNET—on five real network protocol software in the same experimental environment (Ubuntu 18.04 LTS). We quantified the fuzzing results using four metrics related to protocol states and code coverage, and our analysis of these results demonstrated the effectiveness of LLMgSSA.

A. Experimental Setup.

Baseline Fuzzers. AFLNET [5] is a network protocol-oriented fuzzing tool that extends the traditional AFL fuzzer to support stateful network communications. It optimizes the testing process through precise state management and the generation of interaction sequences for the target protocol. NSFuzz [8] is a fuzzing tool for network protocols, focusing on using virtualization technology to isolate the testing environment and provide efficient test execution, thereby safely exploring and exploiting potential vulnerabilities in protocol implementations. CHATAFL is an advanced protocol fuzzing tool based on AFL, which interacts with LLM to acquire

TABLE I
AVERAGE NUMBER OF STATES COVERED BY LLMgSSA AND THE BASELINES.

Subjects	LLMgSSA	CHATAFL	AFLNET	NSFuzz
Exim	20.0	19.5 ↑ 2.6%	14.1 ↑ 41.8%	14.4 ↑ 38.9%
Live555	14.8	14.2 ↑ 4.2%	10.0 ↑ 48.0%	11.7 ↑ 26.5%
ProFTPD	28.9	28.7 ↑ 0.7%	22.6 ↑ 27.9%	24.3 ↑ 18.9%
Fdaapd	12.3	12.1 ↑ 1.7%	8.7 ↑ 41.4%	8.0 ↑ 53.8%
Kamailio	18.0	17.0 ↑ 5.9%	14.0 ↑ 28.6%	15.1 ↑ 19.2%
Avg	18.8	18.3 ↑ 2.7%	13.9 ↑ 35.3%	14.7 ↑ 27.9%

TABLE II
AVERAGE NUMBER OF STATE TRANSITIONS OF LLMgSSA AND THE BASELINES.

Subjects	LLMgSSA	CHATAFL	AFLNET	NSFuzz
Exim	121.9	108.4 ↑ 12.5%	62.7 ↑ 94.4%	69.5 ↑ 75.4%
Live555	183.4	160.0 ↑ 14.6%	83.8 ↑ 118.9%	90.2 ↑ 103.3%
ProFTPD	277.5	246.7 ↑ 12.5%	172.6 ↑ 60.8%	181.2 ↑ 53.1%
Fdaapd	26.5	25.4 ↑ 4.3%	21.4 ↑ 23.8%	20.1 ↑ 31.8%
Kamailio	142.3	130.0 ↑ 9.5%	99.9 ↑ 42.4%	105.3 ↑ 35.1%
Avg	150.3	134.1 ↑ 12.1%	88.1 ↑ 70.6%	93.3 ↑ 61.1%

protocol grammar and uses this information to enhance seeds and guide mutations, thereby improving the coverage and efficiency of fuzzing.

Subject Programs Studied. We have selected five typical network protocols: SMTP, RTSP, FTP, DAAP, and SIP. These protocols cover a broad range of applications and play central roles in email transmission, real-time media stream control, file exchange, music sharing, and real-time communication session management. For each of these protocols, we have chosen mature and widely used implementations provided by ProFuzzBench [6]: Exim, Live555, ProFTPD, forked-daapd (Fdaapd), and Kamailio.

Metrics. The experiment employs four metrics to evaluate the effectiveness of fuzzing in terms of states and code [4], [9]. State Coverage: the number of protocol states covered. It assesses the fuzzer’s exploration of the software’s state space. State Transitions: the number of state transitions generated. A rich set of state transitions during testing helps ensure the robustness of the software during state changes and reveals defects in the transition logic. Line Coverage: the number of lines of software code executed, which is used to evaluate the completeness of the software testing. Branch Coverage: the number of code logic branches executed, which is used to measure the coverage of logical paths.

B. Experimental Results of Fuzzing on States.

Tab. I and Tab. II show that compared to CHATAFL, NSFuzz, and AFLNET, LLMgSSA has improved state coverage and state transitions across five test targets. Valuable states are typically positioned at the program’s centre and serve as springboards to guide protocol software into more states. The perception stage of LLMgSSA identifies central states within the current state space to facilitate state transitions. The intelligent reasoning stage leverages LLM to comprehend the

TABLE III
AVERAGE NUMBER OF BRANCHES COVERED BY LLMgSSA AND THE BASELINES.

Subjects	LLMgSSA	CHATAFL	AFLNET	NSFuzz
Exim	3859.2	3789.4 ↑ 1.8%	3647.6 ↑ 5.8%	3564.3 ↑ 8.3%
Live555	2994.8	2928.4 ↑ 2.3%	2860.2 ↑ 4.7%	2807.6 ↑ 6.7%
ProFTPD	5334.4	5143.3 ↑ 3.7%	4763.0 ↑ 12.0%	4421.8 ↑ 20.6%
Fdaapd	2647.4	2364.8 ↑ 12.0%	2227.1 ↑ 18.9%	2331.3 ↑ 13.6%
Kamailio	11038.7	10064.0 ↑ 9.7%	9404.1 ↑ 17.4%	9758.7 ↑ 13.1%
Avg	5174.9	4858.0 ↑ 6.5%	4580.4 ↑ 13.0%	4576.7 ↑ 13.1%

TABLE IV
AVERAGE NUMBER OF LINES COVERED BY LLMgSSA AND THE BASELINES.

Subjects	LLMgSSA	CHATAFL	AFLNET	NSFuzz
Exim	6347.3	6326.4 ↑ 0.3%	6213.5 ↑ 2.2%	6101.8 ↑ 4.0%
Live555	5908.8	5875.5 ↑ 0.6%	5692.4 ↑ 3.8%	5682.3 ↑ 4.0%
ProFTPD	11195.4	10855.4 ↑ 3.1%	9443.4 ↑ 18.6%	9452.1 ↑ 18.4%
Fdaapd	6986.3	5571.0 ↑ 25.4%	5124.9 ↑ 36.3%	5002.4 ↑ 39.7%
Kamailio	20621.4	20032.1 ↑ 2.9%	19294.5 ↑ 6.9%	18890.2 ↑ 9.2%
Avg	10211.8	9732.1 ↑ 4.9%	9153.7 ↑ 11.6%	9025.8 ↑ 13.1%

entire state machine, identifying covered states most likely to transition into uncovered states, thereby enhancing state coverage. In contrast, the state selection approaches of the baseline fuzzers determine central states based solely on limited fuzzing feedback and fail to choose the most valuable states. The results indicate that LLMgSSA can guide the fuzzer in extensively exploring the state space and state transitions of protocol software, enhancing the ability to discover related errors.

C. Experimental Results of Fuzzing on Code Coverage.

Tab. III and Tab. IV show that compared to CHATAFL, NSFuzz, and AFLNET, LLMgSSA has improved line coverage and branch coverage across five test targets. To increase code coverage, the baseline fuzzers determine valuable states based solely on three types of covered state information: the number of times fuzzed, the number of times selected, and the number of new paths discovered when the state is selected. However, the perception stage of LLMgSSA makes more reliable state selections based on more comprehensive feedback information. Specifically, this stage also considers the number of paths that can reach a covered state and the number of seeds associated with a covered state. Additionally, leveraging LLM’s understanding of the specific protocol’s principles and the corresponding software’s code, the intelligent reasoning stage intelligently infers valuable states based on the current state machine. Finally, the comprehensive decision-making stage synthesizes the results of the previous two stages to determine the states most beneficial for increasing code coverage. The results indicate that LLMgSSA can guide the fuzzer to explore different execution paths in the code more deeply, increasing the likelihood of discovering complex logical errors and boundary condition defects.

V. RELATED WORK

A. State Selection for Protocol Fuzzing.

State selection is a crucial part of protocol fuzzing. Choosing valuable states guides fuzzing to cover more states and code, thereby enhancing the testing of protocol software. AFLNET [5] is the first grey-box fuzzer for protocols. It employs a combination of random selection and a heuristic algorithm for state selection. During the initial phases of fuzzing, random selection is used to explore the state space thoroughly. As sufficient state feedback accumulates, heuristics are employed to explore less visited states or states that have triggered new paths. Subsequent research largely follows AFLNET to select states based on improved heuristic algorithms [1], [4], [8], [10]. However, these approaches, which rely on inadequate fuzzing feedback, possess limited reasoning capabilities and may not effectively identify valuable states in larger state space and complex state transition scenarios.

B. LLM-based Fuzzing.

Benefiting from LLMs' extensive knowledge, powerful understanding, and reasoning capabilities, they have been introduced to fuzzing. By generating more diverse and effective test cases and optimizing various aspects of fuzzing, LLMs have brought revolutionary improvements to the field. TitanFuzz [11] is a method that utilizes LLMs for fuzz testing deep learning libraries, employing LLMs to generate and mutate effective and diverse inputs for deep learning programs. FuzzGPT [12] automates the utilization of historical program features that have triggered vulnerabilities by employing fine-tuning and context learning, thereby enhancing the coverage and detection efficiency of edge cases and potential vulnerabilities in deep learning libraries. Fuzz4All [13] is a universal fuzzing input generation and mutation engine that produces diverse and effective inputs for different systems through automated prompting techniques [14]–[16]. In the aspect of state selection for protocol fuzzing, the field still relies on traditional heuristic algorithms to guide fuzzing, and the introduction of LLMs should bring new opportunities to intelligent state selection in complex scenarios.

VI. CONCLUSION

Protocol fuzzing is one of the main vulnerability mining techniques of network protocol software. Selecting valuable states for stateful protocol fuzzing can cover more states and codes, increasing the chances of uncovering vulnerabilities. Current state selection approaches, which generally use either random selection or heuristics, were limited by inadequate perception of fuzzing feedback and weak reasoning capabilities.

In this work, we have confirmed the potential of LLMs for valuable state selection based on publicly available RFCs and proposed an LLM-guided state selection approach, LLMgSSA. The intelligent state selection of LLMgSSA primarily benefits from two components. The perception component identifies valuable states by perceiving the multidimensional feedback information of each state in the current fuzzing. The intelligent

reasoning component infers valuable states through LLMs' integration and reasoning capabilities regarding extensive knowledge. We have conducted extensive experiments on five widely used real-world protocol software with LLMgSSA and three advanced baseline fuzzers. Experimental results show that LLMgSSA can effectively enhance fuzzing regarding state coverage, state transitions, line coverage, and branch coverage.

ACKNOWLEDGMENT

The work was supported by the Natural Science Foundation of China (61902416, 62302050) and the Natural Science Foundation of Hunan Province in China (2019JJ50729).

REFERENCES

- [1] D. Liu, V.-T. Pham, G. Ernst, T. Murray, and B. I. Rubinstein, "State selection algorithms and their impact on the performance of stateful network protocol fuzzing," in *2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2022, pp. 720–730.
- [2] J. Ba, M. Böhme, Z. Mirzamomen, and A. Roychoudhury, "Stateful greybox fuzzing," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 3255–3272.
- [3] S. Schumilo, C. Aschermann, A. Abbasi, S. Wörner, and T. Holz, "Nyx: Greybox hypervisor fuzzing using fast snapshots and affine types," in *USENIX Security Symposium*, 2021.
- [4] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proceedings of the 31st Annual Network and Distributed System Security Symposium (NDSS)*, 2024.
- [5] V.-T. Pham, M. Böhme, and A. Roychoudhury, "Aflnet: a greybox fuzzer for network protocols," in *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 2020, pp. 460–465.
- [6] R. Natella and V.-T. Pham, "Profuzzbench: A benchmark for stateful protocol fuzzing," in *Proceedings of the 30th ACM SIGSOFT international symposium on software testing and analysis*, 2021, pp. 662–665.
- [7] M. Zalewski, "American fuzzy lop," [Online], <https://github.com/google/AFL>.
- [8] S. Qin, F. Hu, Z. Ma, B. Zhao, T. Yin, and C. Zhang, "Nsfuzz: Towards efficient and state-aware network service fuzzing," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–26, 2023.
- [9] G. Klees, A. Ruef, B. Cooper, S. Wei, and M. Hicks, "Evaluating fuzz testing," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 2123–2138.
- [10] R. Natella, "Stateafl: Greybox fuzzing for stateful network servers," *Empirical Software Engineering*, vol. 27, no. 7, p. 191, 2022.
- [11] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423–435.
- [12] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, "Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt," *arXiv preprint arXiv:2304.02014*, 2023.
- [13] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, "Fuzz4all: Universal fuzzing with large language models," *Proc. IEEE/ACM ICSE*, 2024.
- [14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, pp. 24 824–24 837, 2022.
- [15] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-consistency improves chain of thought reasoning in language models," *arXiv preprint arXiv:2203.11171*, 2022.
- [16] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *International Conference on Data Intelligence and Cognitive Informatics*. Springer, 2023, pp. 387–402.